SCE 672: Advanced Robotics

Term Project Assessment 2: Differential Flatness Pursuit and Motor Voltage Dynamics

Student: Ameen Ur Rehman (202527430) **Marks:** 10
Instructor: Prof. Aamer Iqbal Bhatti
Institution: King Fahd University of Petroleum & Minerals

Problem 1: Visual search, centering and standoff guard

(i) Downward camera pose

The tags lie flat on the floor, so the useful part of the field of view is the floor. A level camera at height h sees a floor point at distance d at a depression angle $\arctan(h/d)$. With the vertical half field of view of the EP stream (about 30°, from $f_y \approx 316$ px at 360 p), a level camera loses the floor in front of it as the robot approaches: points closer than $h/\tan 30^\circ$ fall below the image. Tilting the camera down by β moves the visible floor band closer, so the tag stays in view down to $h/\tan(30^\circ+\beta)$, and it also makes the view of the floor less grazing, which reduces the foreshortening of the tag.

What was measured. The driver was commanded to the pose $(X, Y) = (105, 30)$ mm and the arm read back $x = 0.105$ m, $z = 0.03$ m (the driver names the vertical axis z). Camera frames were captured at this pose and at other arm positions (z from 0.03 to 0.06 m and x from 0.06 to 0.15 m). The frames did not change when only the arm moved, so in this robot the camera is fixed to the chassis and its tilt is not set by the arm pose; the robot description in TF, which attaches the camera to the arm, does not match the hardware and is therefore not used. In the images the horizon (bases of the far walls) lies roughly 100 to 130 px above the image centre, which corresponds to a downward tilt of about $20^\circ \pm 3^\circ$. The gripper claw is visible at the bottom centre of every frame, so a floor tag directly ahead and closer than roughly 0.3 m is hidden by it. The pose (105, 30) mm is a low, retracted arm position; the gripper appears at the bottom of the image, in front of and below the camera, and is meant to stay below the beam of the front Time-of-Flight sensor. The ToF beam clearance was checked from the run log rather than by photograph: the ToF module was enabled and, in the whole Tag 3 run, it never returned a reading below its maximum range, so neither the arm nor the gripper was in its beam. The camera height above the floor was not measured; the tilt above is estimated from the horizon position in the images, and the distance and bearing computed by the node depend only weakly on it.

(ii) Search and centering state machine

Implemented in `state_machine.py` (pure logic, unit tested) and driven by the node at 20 Hz.

- **SEARCHING:** no tag for more than 0.45 s: $v = 0$ and $\omega_z = \pm 0.30$ rad/s, turning toward the side where the tag was last seen (counter-clockwise if never seen).
- **ALIGNING:** tag visible: $v = 0$ and $\omega_z = \mathrm{sat}(K\theta)$ with $K = 0.8\,\mathrm{s}^{-1}$ (limited to 0.12 to 0.40 rad/s) until $|\theta_{\text{bearing}}| \le 0.18$ rad (10°) for 0.3 s.
- **CHASING:** the trajectory of Problem 2 is generated from the distance measured at that moment and followed with a small bearing correction on ω_z. If the bearing exceeds 0.30 rad (hysteresis) the machine returns to ALIGNING. If the tag is lost *before* chasing starts it returns to SEARCHING. If it is lost *during* the chase (a floor tag at close range falls below the bottom of the image, because the camera cannot look straight at the floor near the robot), the planned trajectory is simply continued from its time base, since it already ends at the 20 cm standoff; the camera brake still engages if the tag is visible.

- **ARRIVED:** latched stop (Problem 1 iii).

Why align before translating. The planned path is a straight cubic in the robot frame (boundary conditions $y(0) = y(T) = 0$), so it assumes the tag lies straight ahead. If the robot drove while its heading was off by θ, the open-loop path would end laterally displaced by about $d \sin\theta$ (more than 25 cm at $d = 1.5$ m for 10°), and the tag would drift toward the edge of the image, where it is partly clipped or lost. Rotating first removes the heading error at zero translation, so the planned and executed paths coincide and the tag stays near the image centre.

(iii) Standoff braking

The brake is checked first in every control tick, before any other state logic: if the camera ground distance is $d \le 0.20$ m or the front ToF reads ≤ 0.20 m, the state becomes ARRIVED and $(v, \omega) = (0, 0)$ is commanded immediately and latched.

Problem 2: Differential flatness trajectory

(i) Flatness inversion

With the flat output $z(t) = [x(t), y(t)]^T$ and $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$:

$$v = \sqrt{\dot{x}^2 + \dot{y}^2}, \qquad \omega = \dot{\theta} = \frac{\dot{x}\ddot{y} - \dot{y}\ddot{x}}{\dot{x}^2 + \dot{y}^2}.$$

At standstill ($\dot{x}^2 + \dot{y}^2 \to 0$) the heading is undefined; the implementation returns $\omega = 0$ there.

(ii) Cubic polynomial solution

For each component $z_j(t) = \alpha_0 + \alpha_1 t + \alpha_2 t^2 + \alpha_3 t^3$ the boundary matrix equation is solved with `numpy.linalg.solve` (`solve_flat_polynomial`). For the given boundary values the closed form is

$$\alpha_0 = 0, \quad \alpha_1 = \dot{z}_0, \quad \alpha_2 = \frac{3z_f - 2\dot{z}_0 T}{T^2}, \quad \alpha_3 = \frac{\dot{z}_0 T - 2z_f}{T^3} \qquad (\dot{z}_f = 0).$$

For x: $\dot{x}(0) = 0.06$ m/s, $z_f = d_{\text{tag}} - 0.20$ m. For y all boundary values are zero, hence $y(t) \equiv 0$ and $\omega(t) \equiv 0$: the flat path is a straight line, with speed profile $v(t) = \dot{x}(t)$.

(iii) Time horizon

$T = \max(4.0,\ 1.6\,(d_0 - 0.20)/v_{\max})$ with $v_{\max} = 0.16$ m/s. The mean speed is $z_f/T = v_{\max}/1.6 = 0.10$ m/s, and a cubic that starts near zero speed and ends at zero speed peaks at roughly 1.4 times its mean; computed with the code, the peak is 0.137 m/s for every start distance, always below 0.16 m/s, and $v(T) = 0$:

d_0 (m)	T (s)	α_2 (m/s^2)	α_3 (m/s^3)	peak v (m/s)
1.50	13.0	0.01385	-8.28×10^{-4}	0.137
1.65	14.5	0.01241	-6.66×10^{-4}	0.137
1.80	16.0	0.01125	-5.47×10^{-4}	0.137

Trajectory coefficients for $x(t)$ (computed from the code).

Problem 3: Motor voltage telemetry

(i) Kinematics to wheel rates

With $r = 0.05$ m and $L_c = 0.20$ m, $\omega_1 = \omega_4 = (v_x + L_c\omega_z)/r$ and $\omega_2 = \omega_3 = (v_x - L_c\omega_z)/r$. For example, the in-place search at $\omega_z = 0.30$ rad/s gives $\omega = \pm 1.2$ rad/s, so $V = K_v\omega = \pm 0.456$ V, the

value logged while SEARCHING.

(ii) Armature voltage

$V_i(t) = K_v\omega_i(t) + K_a\,(\omega_i(t) - \omega_i(t-\Delta t))/\Delta t$ with $K_v = 0.38$ and $K_a = 0.06$, evaluated every control tick ($\Delta t = 0.05\,\mathrm{s}$) from the commanded (v, ω_z). The voltages are model outputs computed from the commanded wheel rates; the motor controller itself is not measured. Once the robot halts, $\omega_i = 0$ for two consecutive ticks, so $V_i = 0.000\,\mathrm{V}$ exactly.

(iii) Live table and CSV

Table 1: Chassis motor armature voltage log (target: Tag 3), from `motor_voltage_log.csv`, one row per second.

t (s)	V_1 (V)	V_2 (V)	V_3 (V)	V_4 (V)	**State**
0	0.456	-0.456	-0.456	0.456	`SEARCHING`
1	0.456	-0.456	-0.456	0.456	`SEARCHING`
2	0.456	-0.456	-0.456	0.456	`SEARCHING`
3	0.456	-0.456	-0.456	0.456	`SEARCHING`
4	0.456	-0.456	-0.456	0.456	`SEARCHING`
5	0.456	-0.456	-0.456	0.456	`SEARCHING`
6	0.456	-0.456	-0.456	0.456	`SEARCHING`
7	0.456	-0.456	-0.456	0.456	`SEARCHING`
8	0.456	-0.456	-0.456	0.456	`SEARCHING`
9	0.456	-0.456	-0.456	0.456	`SEARCHING`
10	0.456	-0.456	-0.456	0.456	`SEARCHING`
11	0.456	-0.456	-0.456	0.456	`SEARCHING`
12	-0.569	0.569	0.569	-0.569	`ALIGNING`
13	-0.420	0.420	0.420	-0.420	`ALIGNING`
14	0.707	0.352	0.352	0.707	`CHASING`
15	0.789	0.675	0.675	0.789	`CHASING`
16	0.926	0.880	0.880	0.926	`CHASING`
17	1.007	0.979	0.979	1.007	`CHASING`
18	1.046	1.033	1.033	1.046	`CHASING`
19	1.048	1.017	1.017	1.048	`CHASING`
20	0.975	0.962	0.962	0.975	`CHASING`
21	0.867	0.839	0.839	0.867	`CHASING`
22	0.686	0.666	0.666	0.686	`CHASING`
23	0.440	0.440	0.440	0.440	`CHASING`
24	0.169	0.169	0.169	0.169	`CHASING`
T_{stop}=25.2	0.000	0.000	0.000	0.000	`ARRIVED (trajectory end, tag out of view)`

Run description (tag `tag36h11`, ID 3, black square 0.15 m, standing upright facing the robot; 360 p video; wired USB link; `motor_voltage_log.csv`).

- **SEARCHING** from 0 to 11.4 s: the robot started facing away from the tag and swept in place at 0.30 rad/s, so $V_1 = V_4 = +0.456\,\mathrm{V}$ and $V_2 = V_3 = -0.456\,\mathrm{V}$.
- **ALIGNING** from 11.4 to 13.9 s: the tag came into view and the robot rotated in place (alternating voltage signs while it corrected) until $|\theta| \le 0.18\,\mathrm{rad}$, then settled for 0.3 s.
- **CHASING** from 13.9 to 24.8 s: the cubic trajectory was planned from $d_0 \approx 1.28\,\mathrm{m}$ (bearing 0.15 rad), giving $T = 1.6(1.28 - 0.20)/0.16 = 10.8\,\mathrm{s}$, and the chase took 10.9 s. The peak speed

was 0.137 m/s (limit 0.16). The measured camera distance fell from 1.280 m to 0.351 m (at $t = 22$ s) with a bearing error of about 0.01 rad.

- **ARRIVED** at 24.8 s: below about 0.35 m the upright tag left the top of the image (its centre is above the optical axis), so the last part of the approach ran from the planned trajectory alone, which ends at the 20 cm standoff. The state was logged as `ARRIVED (trajectory end, tag out of view)`. The camera and ToF brakes did not trigger: the camera no longer saw the tag, and the ToF reported no reading below its maximum range (the card is thin). The row at $T_{\text{stop}} = 25.2$ s shows $V_1 = V_2 = V_3 = V_4 = 0.000$ V, as required.

Deviations from the assignment text, stated for transparency: the tag was standing upright rather than lying flat (a flat 15 cm tag is a thin sliver for this camera and cannot be read beyond about 0.7 m), the detector retries at twice the image size to reach the 1.5 to 1.8 m range, and the final 20 cm standoff comes from the dead-reckoned trajectory rather than from a camera reading. **Measured stop distance:** for the run documented above, a tape measure gave a gap of **20 cm** between the front of the robot and the tag after it halted, equal to the target. An earlier trial with the same tag and code (a longer 36 s search) also gave 20 cm. Measurements are by hand (resolution about 1 cm) and the stop is reached by the dead-reckoned trajectory, so it depends on the distance estimate taken at the start of the chase.

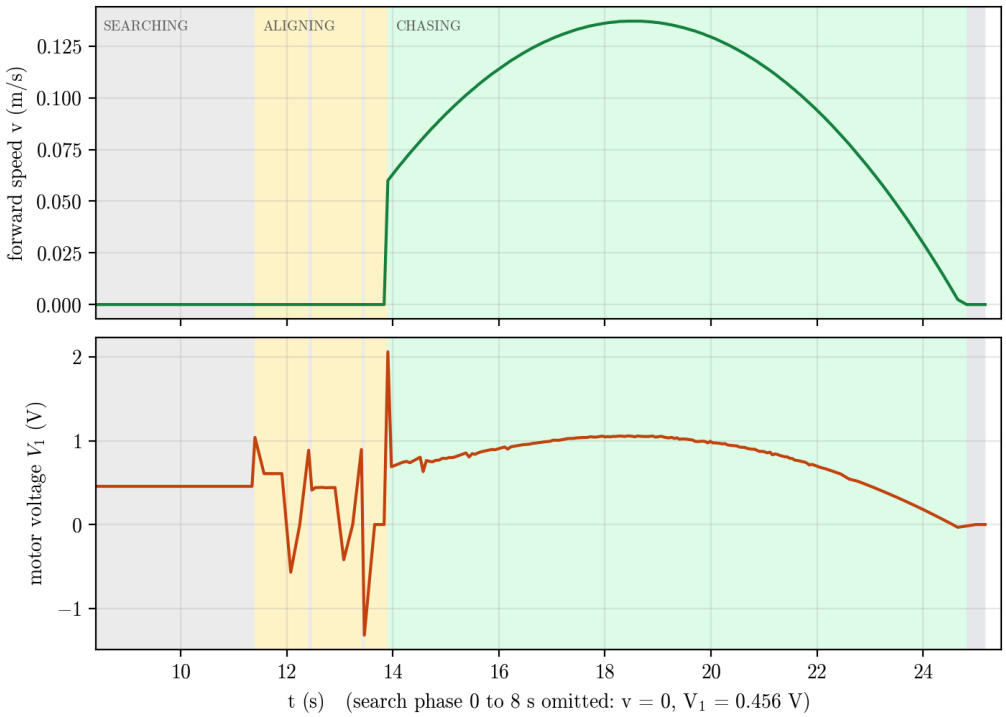


Figure 1: Forward speed $v(t)$ and motor voltage $V_1(t)$ during the Tag 3 run. The first part of the search phase (constant $v = 0$ and $V_1 = 0.456$ V) is omitted from the axis, as stated in the axis label.

Software checks (no robot)

The package has 18 unit tests (boundary conditions, Eq. 1 on a circle, Eq. 4 and Eq. 5 values, state transitions including continuing the trajectory when the tag leaves the image, braking, tag detection and distance/bearing recovery from rendered tags using the EP camera intrinsics). A full ROS-node test with a synthetic camera stream confirmed SEARCHING → ALIGNING → CHASING → ARRIVED with all voltages 0.000 V at the halt. These checks use synthetic data and are not the hardware results above.

Deliverables

- PDF report: this document. Code archive: package `rm_flatness_nav` and `motor_voltage_log.csv` (`assignment2_submission.zip`).
- Demonstration video ($\leq$ 45 s): **[ADD VIDEO LINK HERE]** (sweeping when the tag is hidden, centering on Tag 3, cruising forward and stopping at 20 cm)